

PROJECT REPORT

LIBRARY MANAGEMENT SYSTEM

Submitted By	Hiteshi Bansal
Registration No.	26BCE11580
Program	B.Tech – Computer Science and Engineering
Project Type	Python Programming Project

A beginner-friendly console-based application developed to demonstrate Python programming concepts and basic library operations.

Academic Project Report

CERTIFICATE

This is to certify that **Hiteshi Bansal**, Registration No. **26BCE11580**, student of B.Tech – Computer Science and Engineering, has completed the project titled “**Library Management System**”. The project demonstrates the application of fundamental Python programming concepts to a practical library-management problem.

Student Signature

Faculty Signature

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty members and institution for providing me with the opportunity to work on this project. This project helped me understand the practical use of Python programming concepts such as functions, lists, dictionaries, loops, conditional statements, and user input. I am also thankful to everyone who provided guidance and support during the development and documentation of this project.

Hiteshi Bansal

Reg. No. 26BCE11580

ABSTRACT

The Library Management System is a Python-based console application designed to perform common library operations in a simple and organized manner. The system maintains information about books, including book ID, title, author, availability status, and the student to whom a book is issued. The user can add new books, display the complete collection, search for a book, issue and return books, delete books, and view currently issued books. The project is intended as a beginner-level Computer Science and Engineering application and demonstrates how basic programming concepts can be combined to solve a real-world problem.

Keywords

Python, Library Management, Book Records, Functions, Lists, Dictionaries, Console Application

TABLE OF CONTENTS

1	Introduction
2	Problem Statement
3	Objectives
4	Scope of the Project
5	Technologies and Requirements
6	System Features
7	System Design and Working
8	Algorithm
9	Implementation – Python Code
10	Testing and Sample Output
11	Advantages and Limitations
12	Future Scope
13	Conclusion
14	References

1. INTRODUCTION

Managing books manually can become difficult when the number of books and transactions increases. A computerized library management system provides a structured way to maintain book information and perform routine operations. This project implements a simple console-based solution using Python.

The application uses a list of dictionaries to represent book records. Each record contains a unique book ID, title, author, current status, and issue information. Separate functions are used for each major operation, making the program easier to understand, test, and modify.

2. PROBLEM STATEMENT

The objective is to develop a simple software system that can maintain basic library book records and support everyday operations such as adding, finding, issuing, returning, viewing, and deleting books. The system should provide clear messages to the user and prevent invalid operations such as issuing an already issued book or deleting a book that is currently issued.

3. OBJECTIVES

- To create a simple computerized system for managing library books.
- To store book details in a structured data format.
- To provide search functionality using title or author.
- To implement book issue and return operations.
- To prevent duplicate book IDs and invalid issue/return operations.
- To apply fundamental Python concepts in a practical project.

4. SCOPE OF THE PROJECT

The current version is intended for learning and small-scale library record management. It runs as a console application and keeps data in memory while the program is running. It can be extended later with permanent file/database storage, user authentication, due dates, fine calculation, and a graphical or web interface.

5. TECHNOLOGIES AND REQUIREMENTS

Category	Requirement
Programming Language	Python 3.x
Interface	Command-line / Console
Data Structures	List and Dictionary
Libraries	No external Python libraries required
Operating System	Windows / Linux / macOS
Editor	IDLE, VS Code, PyCharm, or any Python editor

6. SYSTEM FEATURES

Feature	Description
Add Book	Adds a new book after checking that its ID is not already present.
Display All Books	Shows the details and current status of every book.
Search Book	Searches by part of a book title or author name.
Issue Book	Changes a book's status to Issued and records the student name.
Return Book	Changes an issued book back to Available.
Delete Book	Removes an available book from the collection.
View Issued Books	Displays books that are currently issued.
Exit	Closes the application safely.

7. SYSTEM DESIGN AND WORKING

The system starts by creating an initial collection of books. The main() function displays a menu in a loop. According to the user's choice, it calls the corresponding function. Each function performs one operation and then returns control to the main menu.

Basic Flow

1. Start application
2. Load initial book records
3. Display main menu
4. Read user's choice
5. Perform selected operation
6. Show result
7. Return to main menu
8. Exit when the user selects option 8

8. ALGORITHM

1. Start.
2. Create a list containing book records.
3. Display the Library Management System menu.
4. Accept the user's choice.
5. If choice is Add Book, accept book details and append a new record.
6. If choice is Display Books, print all records.

7. If choice is Search Book, compare the search text with titles and authors.
8. If choice is Issue Book, check availability and store the student's name.
9. If choice is Return Book, change the status to Available.
10. If choice is Delete Book, remove the selected available record.
11. If choice is View Issued Books, display all issued records.
12. If choice is Exit, terminate the program.
13. Otherwise, display an invalid-choice message.
14. Repeat the menu until the user chooses Exit.
15. Stop.

9. IMPLEMENTATION – PYTHON CODE

```
# LIBRARY MANAGEMENT SYSTEM

books = [
    {"id": 1, "title": "Python Programming",
     "author": "R. Nageswara Rao", "status": "Available", "issued_to": ""},
    {"id": 2, "title": "Data Structures",
     "author": "Seymour Lipschutz", "status": "Available", "issued_to": ""},
    {"id": 3, "title": "Computer Fundamentals",
     "author": "P. K. Sinha", "status": "Available", "issued_to": ""}
]

def add_book():
    print("\n===== ADD BOOK =====")
    book_id = int(input("Enter Book ID: "))

    for book in books:
        if book["id"] == book_id:
            print("Book ID already exists!")
            return

    title = input("Enter Book Title: ")
    author = input("Enter Author Name: ")

    new_book = {
        "id": book_id,
        "title": title,
        "author": author,
        "status": "Available",
        "issued_to": ""
    }

    books.append(new_book)
    print("Book added successfully!")

def display_books():
    print("\n===== ALL BOOKS =====")

    if len(books) == 0:
        print("No books available.")
        return

    for book in books:
        print("-----")
        print("Book ID      :", book["id"])
        print("Title       :", book["title"])
        print("Author      :", book["author"])
        print("Status      :", book["status"])

        if book["status"] == "Issued":
            print("Issued To   :", book["issued_to"])

def search_book():
    print("\n===== SEARCH BOOK =====")
    search = input("Enter book title or author: ").lower()
    found = False

    for book in books:
        if (search in book["title"].lower() or
            search in book["author"].lower()):
            print("\nBook Found!")
            print("Book ID      :", book["id"])
            print("Title       :", book["title"])
            print("Author      :", book["author"])
            print("Status      :", book["status"])
            if book["status"] == "Issued":
                print("Issued To   :", book["issued_to"])
            found = True

    if not found:
        print("Book not found.")

def issue_book():
```



```

print("\n===== ISSUE BOOK =====")
book_id = int(input("Enter Book ID to issue: "))

for book in books:
    if book["id"] == book_id:
        if book["status"] == "Issued":
            print("Book is already issued.")
            return

        student_name = input("Enter student name: ")
        book["status"] = "Issued"
        book["issued_to"] = student_name
        print("Book issued successfully!")
        return

print("Book ID not found.")

def return_book():
    print("\n===== RETURN BOOK =====")
    book_id = int(input("Enter Book ID to return: "))

    for book in books:
        if book["id"] == book_id:
            if book["status"] == "Available":
                print("This book is already available.")
                return

            book["status"] = "Available"
            book["issued_to"] = ""
            print("Book returned successfully!")
            return

    print("Book ID not found.")

def delete_book():
    print("\n===== DELETE BOOK =====")
    book_id = int(input("Enter Book ID to delete: "))

    for book in books:
        if book["id"] == book_id:
            if book["status"] == "Issued":
                print("Cannot delete an issued book.")
                return

            books.remove(book)
            print("Book deleted successfully!")
            return

    print("Book ID not found.")

def issued_books():
    print("\n===== ISSUED BOOKS =====")
    found = False

    for book in books:
        if book["status"] == "Issued":
            print("-----")
            print("Book ID      :", book["id"])
            print("Title       :", book["title"])
            print("Author      :", book["author"])
            print("Issued To   :", book["issued_to"])
            found = True

    if not found:
        print("No books are currently issued.")

def main():
    while True:
        print("\n=====")
        print("      LIBRARY MANAGEMENT SYSTEM")
        print("=====")
        print("1. Add Book")
        print("2. Display All Books")
        print("3. Search Book")
        print("4. Issue Book")

```

```

print("5. Return Book")
print("6. Delete Book")
print("7. View Issued Books")
print("8. Exit")
print("=====")

choice = input("Enter your choice (1-8): ")

if choice == "1":
    add_book()
elif choice == "2":
    display_books()
elif choice == "3":
    search_book()
elif choice == "4":
    issue_book()
elif choice == "5":
    return_book()
elif choice == "6":
    delete_book()
elif choice == "7":
    issued_books()
elif choice == "8":
    print("\nThank you for using Library Management System!")
    break
else:
    print("Invalid choice! Please enter a number from 1 to 8.")

main()

```

10. TESTING AND SAMPLE OUTPUT

Test Case	Input / Action	Expected Result
1	Choose 1 and enter a new unique ID	Book is added successfully
2	Choose 2	All books and their statuses are displayed
3	Search an existing title	Matching book details are displayed
4	Issue an available book	Status changes to Issued
5	Issue an already issued book	System rejects the operation
6	Return an issued book	Status changes to Available
7	Delete an available book	Book is removed
8	Choose 8	Program exits

Sample Console Output

```

=====
LIBRARY MANAGEMENT SYSTEM
=====
1. Add Book
2. Display All Books
3. Search Book
4. Issue Book
5. Return Book
6. Delete Book
7. View Issued Books
8. Exit
=====
Enter your choice (1-8): 4

===== ISSUE BOOK =====
Enter Book ID to issue: 1
Enter student name: Hiteshi
Book issued successfully!

```

11. ADVANTAGES AND LIMITATIONS

Advantages

- Simple and easy-to-use console interface.
- Uses modular functions for better organization.
- Searches books quickly by title or author.
- Prevents duplicate book IDs.
- Prevents issuing an already issued book.
- Requires no external libraries.

Limitations

- Data is stored only while the program is running.
- There is no login or role-based access control.
- The system does not calculate due dates or fines.
- It is a console application rather than a graphical/web application.
- The current version is intended for small-scale use.

12. FUTURE SCOPE

- Store records permanently using text files, CSV, SQLite, or another database.
- Add student registration and login functionality.
- Add issue dates, due dates, and automatic fine calculation.
- Develop an admin panel for managing books and users.
- Create a GUI using Tkinter or a web interface using Flask.
- Add book categories, authors, publishers, and advanced search filters.
- Generate reports of issued, returned, and overdue books.

13. CONCLUSION

The Library Management System successfully demonstrates how Python can be used to solve a basic real-world record-management problem. The application provides the core operations required for managing a small collection of library books and applies important programming concepts including functions, loops, conditional statements, lists, dictionaries, string operations, and user input. The project also provides a strong foundation for future enhancements such as permanent storage, authentication, fine calculation, and a graphical or web-based interface.

14. REFERENCES

- Python Software Foundation. Python Documentation.

- Python 3.x language concepts: functions, lists, dictionaries, loops, and conditional statements.
- Course notes and classroom material on Python programming.